

LangGraph: The Complete Tutorial

From your first graph to multi-agent systems -- with every example tested against the real library

Generated on August 22, 2026 at 03:52 PM

Written and verified against langgraph 1.2.11 and langchain 1.6.0 (current as of this writing)

This tutorial takes you from "what is LangGraph" all the way to multi-agent systems, human-in-the-loop workflows, and persistent memory. Every single code example in this document was actually run against the real, currently-installed LangGraph library before being included here -- not guessed from memory -- so what you see is what actually happens when you run it (sample output is shown under most examples). Start at the top if you're new to LangGraph; jump to Section 9 onward if you already know the basics and want the advanced material.

Index

Index	1
1. What Is LangGraph, and Why Use It?	2
2. Installation & Setup	2
3. Core Concepts, In Plain Terms	3
4. Beginner Example 1: "Hello, World" (One Node)	4
5. Beginner Example 2: Chaining Multiple Nodes	4
6. Beginner Example 3: Conditional Edges (Branching)	4
7. How State Updates Merge: Reducers	5
8. Cycles: Why LangGraph Isn't Just a DAG	6
9. Building a Tool-Calling Agent by Hand	8
10. Prebuilt Agents: create_agent	9
11. Streaming Output	10
12. Checkpointing & Memory (Threads)	11
13. Human-in-the-Loop: interrupt() and Command	12
14. Parallel Fan-Out / Fan-In	12
15. Subgraphs: Graphs Inside Graphs	13
16. Multi-Agent Systems: The Supervisor Pattern	15
17. Capstone: Putting It All Together	17
18. Cheat Sheet / Quick Reference	19
19. Common Pitfalls & Best Practices	20
References	21

1. What Is LangGraph, and Why Use It?

In one sentence: LangGraph is a Python framework for building AI applications as an explicit graph -- a set of steps (*nodes*) connected by rules about what runs next (*edges*), all sharing one piece of data (*state*) that flows through the whole run.

You could write an agent as a plain Python while-loop calling an LLM over and over. LangGraph exists because once that loop needs to branch, retry, pause for a human, remember past conversations, run steps in parallel, or hand off work between multiple "agents", hand-rolling all of that gets messy fast. LangGraph gives you: automatic state merging across steps, built-in persistence ("checkpointing") so a run can pause and resume exactly where it left off, native support for cycles (loops) as a first-class citizen -- not a hack -- and a visual, inspectable structure that's the same whether your graph has 3 nodes or 30.

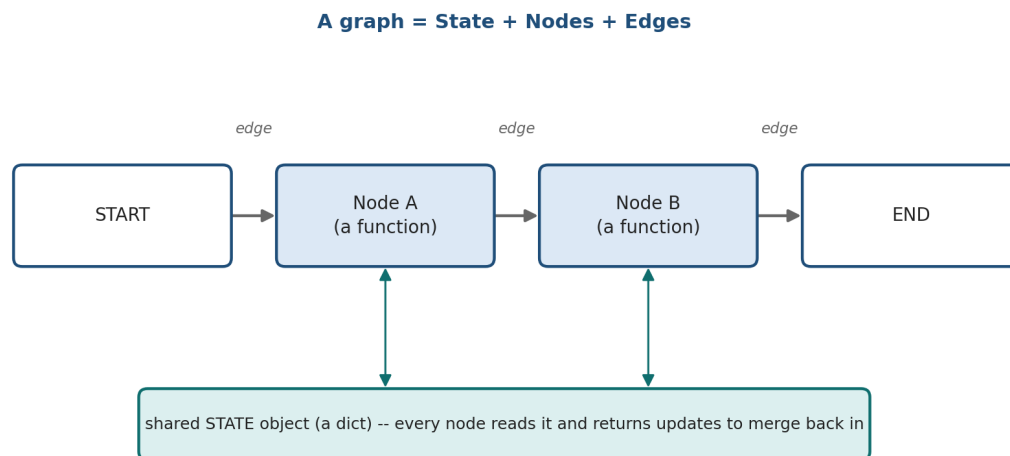


Figure 1. Every LangGraph app is the same three ingredients: State, Nodes, and Edges.

- **State** -- a Python dict-like object (usually a TypedDict) describing everything the graph needs to remember: messages so far, counters, results, whatever your app needs.
- **Node** -- a plain Python function. It receives the current state and returns a *partial update* -- only the keys it wants to change.
- **Edge** -- connects one node to the next. A *normal edge* always goes the same way; a *conditional edge* picks the next node at runtime based on the state.

LangGraph is built by the LangChain team and is commonly used together with langchain-core (for chat message types and tool definitions), but the graph engine itself has no hard dependency on any particular model provider.

2. Installation & Setup

```
pip install langgraph langchain langchain-core

# Requires Python 3.10 or higher (LangGraph v1.0 dropped 3.9 support).
# This tutorial was written and tested against:
#   langgraph      1.2.11
#   langchain       1.6.0
#   langchain-core  1.6.0
```

You'll also need an actual chat model to talk to in real usage (e.g. `langchain-anthropic` or `langchain-openai`), but every example in this tutorial that needs a model uses a small local *fake* model instead, so you can run every single snippet here with zero API keys and zero cost, purely to learn the mechanics of LangGraph itself.

3. Core Concepts, In Plain Terms

An everyday analogy: think of a graph like a flowchart for a customer support call. The *state* is the customer's case file, updated as the call goes on. Each *node* is one step an agent takes ("look up account", "check refund policy", "escalate"). The *edges* are the rules for what happens next ("if the refund is under \$50, approve it automatically; otherwise, escalate"). LangGraph is just a precise, code-based way to define that flowchart and actually run it.

- `StateGraph(StateSchema)` -- start building a graph, given a type describing your state's shape.
- `graph.add_node(name, function)` -- register a step.
- `graph.add_edge(a, b)` -- always go from node a to node b.
- `graph.add_conditional_edges(a, router_fn, mapping)` -- go from a to whichever node `router_fn(state)` returns.
- `graph.compile()` -- turn the builder into a runnable app.
- `app.invoke(initial_state) / app.stream(...)` -- run it.

START and **END** are special built-in markers, not real nodes -- `add_edge(START, "my_node")` means "begin here"; an edge into **END** means "the run is finished".

4. Beginner Example 1: "Hello, World" (One Node)

The smallest possible graph: one node that transforms the state, wired directly from START to END.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    greeting: str

def say_hello(state: State) -> State:
    return {"greeting": f"Hello, {state['greeting']}!"}

graph = StateGraph(State)
graph.add_node("hello", say_hello)
graph.add_edge(START, "hello")
graph.add_edge("hello", END)

app = graph.compile()
print(app.invoke({"greeting": "world"}))

{'greeting': 'Hello, world!'}
```

Notice the node function doesn't mutate state -- it returns a **new dict with just the keys it wants to change**. LangGraph merges that into the running state for you. This matters a lot once you have more than one field in your state.

5. Beginner Example 2: Chaining Multiple Nodes

Add a second node and an edge between them -- now the state flows through two steps in order.

```
class State(TypedDict):
    text: str

def shout(state: State) -> State:
    return {"text": state["text"].upper()}

def exclaim(state: State) -> State:
    return {"text": state["text"] + "!!!" }

graph = StateGraph(State)
graph.add_node("shout", shout)
graph.add_node("exclaim", exclaim)
graph.add_edge(START, "shout")
graph.add_edge("shout", "exclaim")    # shout runs, THEN exclaim runs
graph.add_edge("exclaim", END)

app = graph.compile()
print(app.invoke({"text": "langgraph is fun"}))

{'text': 'LANGGRAPH IS FUN!!!'}
```

6. Beginner Example 3: Conditional Edges (Branching)

A *router function* looks at the state and returns the name of whichever node should run next. This is how an agent "makes a decision".

add_conditional_edges(): one router, many possible next nodes

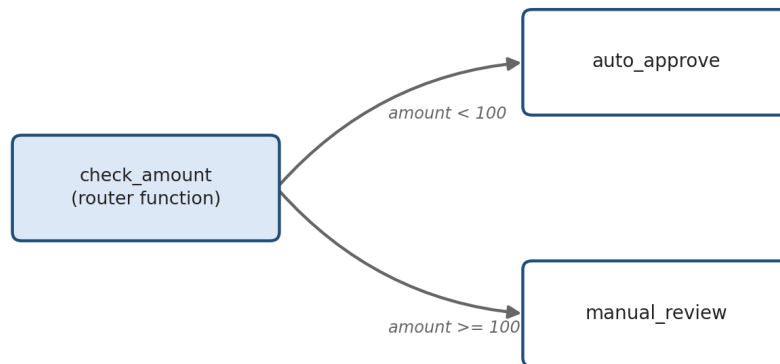


Figure 2. `add_conditional_edges` wires one router to several possible destinations.

```
from typing import Literal

class State(TypedDict):
    amount: float
    route: str

def check_amount(state: State) -> State:
    return {} # a node can also do nothing to the state, just pass through

def route_by_amount(state: State) -> Literal["auto_approve", "manual_review"]:
    return "auto_approve" if state["amount"] < 100 else "manual_review"

def auto_approve(state: State) -> State:
    return {"route": "auto-approved"}

def manual_review(state: State) -> State:
    return {"route": "sent to a human reviewer"}

graph = StateGraph(State)
graph.add_node("check_amount", check_amount)
graph.add_node("auto_approve", auto_approve)
graph.add_node("manual_review", manual_review)
graph.add_edge(START, "check_amount")
graph.add_conditional_edges(
    "check_amount", route_by_amount,
    {"auto_approve": "auto_approve", "manual_review": "manual_review"},
)
graph.add_edge("auto_approve", END)
graph.add_edge("manual_review", END)

app = graph.compile()
print(app.invoke({"amount": 42, "route": ""}))
print(app.invoke({"amount": 4200, "route": ""}))

{'amount': 42, 'route': 'auto-approved'}
{'amount': 4200, 'route': 'sent to a human reviewer'}
```

7. How State Updates Merge: Reducers

By default, when a node returns `{"count": 5}`, LangGraph simply **overwrites** the old value of `count`. But sometimes you want to *accumulate* instead -- e.g. append a new chat message to a growing list rather than

replace the whole list. That's what a **reducer** is for: a function attached to a field that says how to combine the old value with the new update.

```
import operator
from typing import Annotated

# operator.add as a reducer: new list items get APPENDED, not overwritten
class State(TypedDict):
    count: int
    log: Annotated[list[str], operator.add]

def increment(state: State) -> State:
    return {"count": state["count"] + 1, "log": [f"count is now {state['count']} + 1"]}
```

The most common reducer you'll use is `add_messages`, built into LangGraph specifically for chat history: it appends new messages, and if a new message reuses an existing message's `id`, it replaces that message instead of duplicating it (handy for streaming edits).

```
from langgraph.graph import add_messages
from langchain_core.messages import HumanMessage, AIMessage

class State(TypedDict):
    messages: Annotated[list, add_messages]

def add_greeting(state: State) -> State:
    return {"messages": [AIMessage(content="hi there!")]}
```

```
graph = StateGraph(State)
graph.add_node("add_greeting", add_greeting)
graph.add_edge(START, "add_greeting")
graph.add_edge("add_greeting", END)

app = graph.compile()
result = app.invoke({"messages": [HumanMessage(content="hello")]}))
for m in result["messages"]:
    print(type(m).__name__, "->", m.content)
```

```
HumanMessage -> hello
AIMessage -> hi there!
```

Without a reducer, that second invoke would have replaced messages entirely, losing the "hello" -- this is the single most common beginner bug when building chat agents in LangGraph.

8. Cycles: Why LangGraph Isn't Just a DAG

A plain chain (A then B then C) is a one-way pipeline. Real agents need to *loop*: keep calling a tool until you have enough information, or keep retrying until something succeeds. LangGraph makes this natural: a conditional edge can route **back to a node you've already visited**, and the graph will keep going around until the router decides to send it to END.

```

class State(TypedDict):
    count: int
    log: Annotated[list[str], operator.add]

def increment(state: State) -> State:
    return {"count": state["count"] + 1, "log": [f"count is now {state['count']} + 1"]}

def should_continue(state: State) -> Literal["increment", "__end__"]:
    return "increment" if state["count"] < 3 else "__end__"

graph = StateGraph(State)
graph.add_node("increment", increment)
graph.add_edge(START, "increment")
graph.add_conditional_edges(
    "increment", should_continue, {"increment": "increment", "__end__": END},
)

app = graph.compile()
print(app.invoke({"count": 0, "log": []}))

{'count': 3, 'log': ['count is now 1', 'count is now 2', 'count is now 3']}

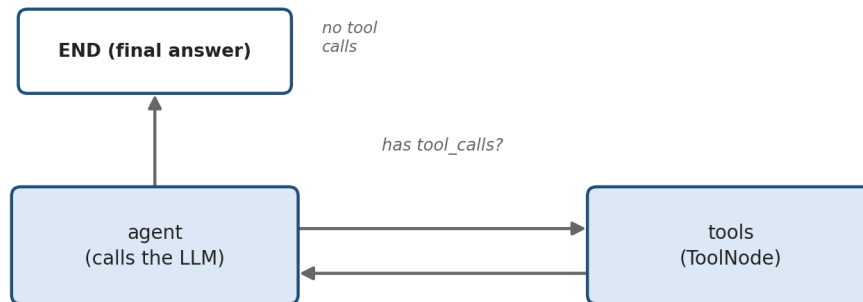
```

This is the exact same mechanism a tool-calling agent uses -- "keep looping while there's more work to do" -- just with a counter instead of an LLM deciding when to stop. Next section builds the real thing.

9. Building a Tool-Calling Agent by Hand

Now the real pattern behind almost every LLM agent: an agent node calls the model; if the model's response contains tool calls, a `tools` node executes them and loops back; if not, the graph ends. LangGraph ships two prebuilt helpers for exactly this: `ToolNode` (runs whatever tools the model asked for) and `tools_condition` (the router: "does the last message have tool_calls?").

Cycles: what plain DAG-based chains can't do



tools_condition routes agent -> tools OR agent -> END each time around the loop

Figure 3. The agent <-> tools loop -- this is the ReAct pattern, built out of a cycle.

```

from langgraph.prebuilt import ToolNode, tools_condition
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f'It's sunny in {city}.'

class State(TypedDict):
    messages: Annotated[list, add_messages]

def call_model(state: State) -> State:
    model_with_tools = model.bind_tools([get_weather]) # `model` = your real chat model
    response = model_with_tools.invoke(state["messages"])
    return {"messages": [response]}

graph = StateGraph(State)
graph.add_node("agent", call_model)
graph.add_node("tools", ToolNode([get_weather]))
graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", tools_condition) # -> "tools" or END, built in
graph.add_edge("tools", "agent") # loop back after running tools

app = graph.compile()
result = app.invoke({"messages": [HumanMessage(content="What's the weather in Paris?")]})
for m in result["messages"]:
    print(type(m).__name__, "->", getattr(m, "content", None))

```

```

HumanMessage -> What's the weather in Paris?
AIMessage ->
ToolMessage -> It's sunny in Paris.
AIMessage -> It's sunny in Paris right now.

```

The graph went agent -> tools -> agent automatically: the first AIMessage carried a tool call (empty content, a tool_calls field instead), tools_condition sent it to the tools node, which ran get_weather and produced a ToolMessage, which looped back to agent for a final answer.

10. Prebuilt Agents: create_agent

The loop above is common enough that LangGraph ships it prebuilt. **Important, current detail:** as of LangGraph v1.0, the old langgraph.prebuilt.create_react_agent is deprecated in favor of langchain.agents.create_agent, which runs on LangGraph under the hood but adds a more flexible middleware system. The old import still works for now but will be removed in a future release.

```

# OLD (deprecated since LangGraph v1.0, still works but will be removed):
# from langgraph.prebuilt import create_react_agent
# agent = create_react_agent(model, tools=[get_weather])

# NEW (recommended):
from langchain.agents import create_agent

agent = create_agent(model, tools=[get_weather])
result = agent.invoke({"messages": [HumanMessage(content="Weather in Paris?")]})
print(result["messages"][-1].content)

```

```

It's sunny in Paris right now.

```

That's the entire agent loop from Section 9 in three lines. Use create_agent for standard tool-calling agents, and drop down to a hand-built graph (Section 9's pattern) when you need custom routing logic the prebuilt agent

doesn't support -- e.g. a step between tool calls, or multiple different "kinds" of agent loop in one graph.

11. Streaming Output

Instead of waiting for the whole graph to finish, `.stream()` yields output as each node completes. `stream_mode="updates"` yields just the dict each node returned; other modes ("values", "messages") yield the full state or token-by-token LLM output.

```
for chunk in app.stream({"text": "streaming demo"}, stream_mode="updates"):
    print("stream chunk:", chunk)
```

```
stream chunk: {'shout': {'text': 'STREAMING DEMO'}}
stream chunk: {'exclaim': {'text': 'STREAMING DEMO!!!'}}
```

This is exactly why the "one node = one function" model matters: each stream chunk is labeled with the name of the node that produced it, so a UI can show "thinking...", "searching...", "writing answer..." as the graph runs.

12. Checkpointing & Memory (Threads)

So far, every `.invoke()` has started from a blank slate. Real chat apps need memory across turns. LangGraph does this with a **checkpoint**: attach one at `compile()` time, then pass a `thread_id` in the config on every call. Each distinct `thread_id` gets its own saved history; the same `thread_id` automatically picks up where it left off.

Checkpoint + thread_id: one saved history PER thread

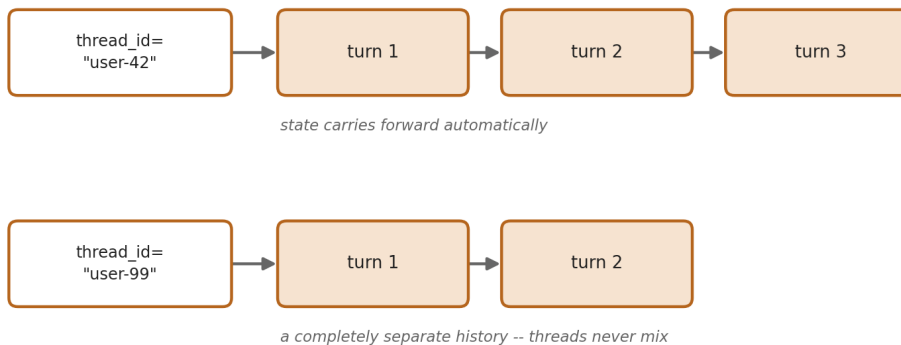


Figure 4. Each `thread_id` is a completely independent, persisted conversation history.

```
from langgraph.checkpoint.memory import InMemorySaver

class State(TypedDict):
    messages: Annotated[list, add_messages]

def echo(state: State) -> State:
    last = state["messages"][-1].content
    return {"messages": [AIMessage(content=f"You said: {last}")]}}

graph = StateGraph(State)
graph.add_node("echo", echo)
graph.add_edge(START, "echo")
graph.add_edge("echo", END)

checkpointer = InMemorySaver()
app = graph.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "user-42"}}
app.invoke({"messages": [HumanMessage(content="first message")]}, config)
app.invoke({"messages": [HumanMessage(content="second message")]}, config)

state = app.get_state(config)
print("messages remembered across turns:", len(state.values["messages"]))

messages remembered across turns: 4
```

4 messages: ["first message", "You said: first message", "second message", "You said: second message"] -- the second `.invoke()` call didn't need to resend any history; the checkpointer supplied it automatically because the `thread_id` matched. `InMemorySaver` is for local dev/testing only -- it's an alias of `MemorySaver` and loses everything on process restart. For production, LangGraph ships Postgres and SQLite checkpointer backends that persist to disk/a real database.

13. Human-in-the-Loop: interrupt() and Command

Some steps shouldn't happen without a human saying yes -- sending an email, approving a refund, deleting data. LangGraph's `interrupt()` pauses the graph *mid-node*, saves everything via the checkpoint, and returns control to your program. Later, calling `.invoke()` again with `Command(resume=...)` picks that exact node back up with the human's answer plugged in. This requires a checkpointer -- there has to be somewhere to save the paused state.

```
from langgraph.types import Command, interrupt

class State(TypedDict):
    request: str
    approved: bool

def request_approval(state: State) -> State:
    decision = interrupt({"question": f"Approve this? {state['request']}"})
    return {"approved": decision}

graph = StateGraph(State)
graph.add_node("request_approval", request_approval)
graph.add_edge(START, "request_approval")
graph.add_edge("request_approval", END)

app = graph.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "approval-1"}}

out = app.invoke({"request": "refund $500", "approved": False}, config)
print("paused:", out)

out2 = app.invoke(Command(resume=True), config) # human said yes
print("resumed:", out2)

paused: {'request': 'refund $500', 'approved': False,
        '__interrupt__': [Interrupt(value={'question': 'Approve this? refund $500'}, ...)]}
resumed: {'request': 'refund $500', 'approved': True}
```

The first call returns immediately with an `__interrupt__` key -- your app would show that question to a human (in a UI, a Slack message, whatever). Whatever value you pass to `Command(resume=...)` becomes `interrupt()`'s return value *inside the paused node*, and execution continues from exactly there -- not from the top of the graph.

14. Parallel Fan-Out / Fan-In

Point two edges out of the same node (or out of START) and both branches run concurrently; point them both into the same downstream node and LangGraph waits for all of them before continuing. This needs a reducer on the shared field, since two nodes will both try to write to it in the same step.

```

class State(TypedDict):
    topic: str
    angles: Annotated[list[str], operator.add]  # reducer -- both branches APPEND

def angle_a(state: State) -> State:
    return {"angles": [f"[history angle] {state['topic']}"]}

def angle_b(state: State) -> State:
    return {"angles": [f"[economics angle] {state['topic']}"]}

def combine(state: State) -> State:
    return {}  # everything is already merged in `angles` by the time we get here

graph = StateGraph(State)
graph.add_node("angle_a", angle_a)
graph.add_node("angle_b", angle_b)
graph.add_node("combine", combine)
graph.add_edge(START, "angle_a")  # both fire from START
graph.add_edge(START, "angle_b")
graph.add_edge("angle_a", "combine")  # both feed into combine
graph.add_edge("angle_b", "combine")
graph.add_edge("combine", END)

app = graph.compile()
print(app.invoke({"topic": "trade routes", "angles": []}))

{'topic': 'trade routes',
 'angles': ['[history angle] trade routes', '[economics angle] trade routes']}

```

Without the operator .add reducer, this would raise an error (or silently drop one branch's result, depending on version) because two nodes wrote to angles in the same step with no rule for combining them.

15. Subgraphs: Graphs Inside Graphs

A compiled graph is itself a valid node in another graph. This is how you keep large systems manageable -- build and test a self-contained piece (say, a whole "research" sub-agent) on its own, then drop it into a bigger graph as a single node.

```

class InnerState(TypedDict):
    value: int

def double(state: InnerState) -> InnerState:
    return {"value": state["value"] * 2}

inner = StateGraph(InnerState)
inner.add_node("double", double)
inner.add_edge(START, "double")
inner.add_edge("double", END)
inner_app = inner.compile()  # a fully working graph on its own

class OuterState(TypedDict):
    value: int

outer = StateGraph(OuterState)
outer.add_node("doubling_subgraph", inner_app)  # <-- a compiled graph used AS a node
outer.add_edge(START, "doubling_subgraph")
outer.add_edge("doubling_subgraph", END)
outer_app = outer.compile()
print(outer_app.invoke({"value": 5}))

{'value': 10}

```

This works cleanly here because both states share the same shape (value). For subgraphs with a different internal state shape than the parent, you'd wrap the subgraph in a small adapter node that translates between the two.

16. Multi-Agent Systems: The Supervisor Pattern

Once one agent isn't enough -- you want a "researcher" and a "writer" with different tools and prompts -- the most common structure is a **supervisor**: one router node that looks at the task and decides which specialized worker node should handle it. This is just conditional edges again, at the level of whole agents instead of individual actions.

Supervisor pattern: one router node, many worker nodes

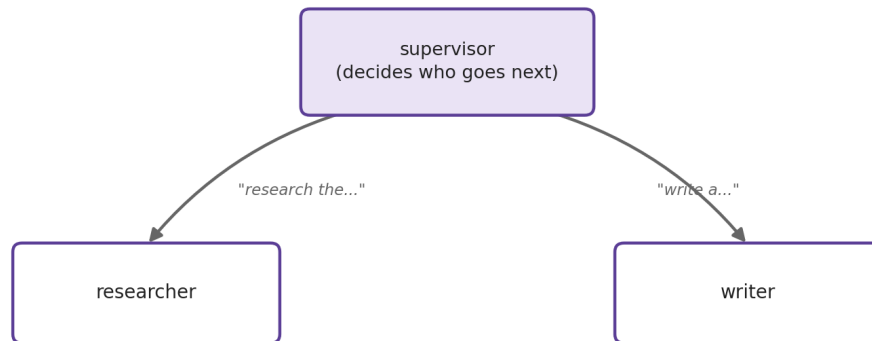


Figure 5. A supervisor node routes work to specialized worker nodes.

```
class State(TypedDict):
    task: str
    result: str
    next_agent: str

def supervisor(state: State) -> State:
    next_agent = "researcher" if "research" in state["task"] else "writer"
    return {"next_agent": next_agent}

def route_supervisor(state: State) -> str:
    return state["next_agent"]

def researcher(state: State) -> State:
    return {"result": f"[researcher] gathered facts for: {state['task']}"}

def writer(state: State) -> State:
    return {"result": f"[writer] drafted text for: {state['task']}"}

graph = StateGraph(State)
graph.add_node("supervisor", supervisor)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_edge(START, "supervisor")
graph.add_conditional_edges(
    "supervisor", route_supervisor,
    {"researcher": "researcher", "writer": "writer"},
)
graph.add_edge("researcher", END)
graph.add_edge("writer", END)

app = graph.compile()
print(app.invoke({"task": "research the history of chess", "result": "", "next_agent": ""}))
print(app.invoke({"task": "write a poem about autumn", "result": "", "next_agent": ""}))
```

```
{'task': 'research the history of chess', 'next_agent': 'researcher',  
  'result': '[researcher] gathered facts for: research the history of chess'}  
{'task': 'write a poem about autumn', 'next_agent': 'writer',  
  'result': '[writer] drafted text for: write a poem about autumn'}
```

In production, each worker is usually itself a small agent graph (Section 9's loop), and control returns to the supervisor after each worker finishes so it can decide the next step -- add an edge from each worker back to the supervisor instead of to END, with a routing rule for "we're done, go to END". LangChain also publishes a ready-made langgraph-supervisor package implementing this pattern with richer handoff semantics via the Command object, plus a **"swarm"** variant where agents hand off directly to each other with no central supervisor -- worth knowing both terms exist if this comes up in an interview.

17. Capstone: Putting It All Together

An expense-approval agent that combines four things from this tutorial: a **tool call** to check company policy, a **conditional edge** to route based on the result, **human-in-the-loop** for anything over policy, and a **checkpointer** so the pause/resume actually works.

```
@tool
def check_policy(amount: float) -> str:
    """Check company policy for an expense amount."""
    return "needs_review" if amount > 500 else "auto_ok"

class ExpenseState(TypedDict):
    amount: float
    memo: str
    policy_result: str
    decision: str

def check_expense(state: ExpenseState) -> ExpenseState:
    result = check_policy.invoke({"amount": state["amount"]})
    return {"policy_result": result}

def route_after_check(state: ExpenseState) -> Literal["auto_approve", "ask_human"]:
    return "auto_approve" if state["policy_result"] == "auto_ok" else "ask_human"

def auto_approve(state: ExpenseState) -> ExpenseState:
    return {"decision": f"Auto-approved ${state['amount']} for '{state['memo']}'."}

def ask_human(state: ExpenseState) -> ExpenseState:
    human_says = interrupt({
        "question": f"Expense of ${state['amount']} for '{state['memo']}' exceeds "
                    f"policy. Approve?",
    })
    verdict = "Approved" if human_says else "Rejected"
    return {"decision": f"{verdict} by human reviewer: ${state['amount']} for '{state['memo']}'."}

graph = StateGraph(ExpenseState)
graph.add_node("check_expense", check_expense)
graph.add_node("auto_approve", auto_approve)
graph.add_node("ask_human", ask_human)
graph.add_edge(START, "check_expense")
graph.add_conditional_edges(
    "check_expense", route_after_check,
    {"auto_approve": "auto_approve", "ask_human": "ask_human"},
)
graph.add_edge("auto_approve", END)
graph.add_edge("ask_human", END)

app = graph.compile(checkpointer=InMemorySaver())

# Case 1: small amount, no human needed
cfg1 = {"configurable": {"thread_id": "expense-1"}}
out1 = app.invoke({"amount": 120, "memo": "team lunch", "policy_result": "", "decision": ""}, cfg1)
print("Case 1:", out1["decision"])

# Case 2: large amount -- pauses for a human
cfg2 = {"configurable": {"thread_id": "expense-2"}}
out2 = app.invoke({"amount": 900, "memo": "conference travel", "policy_result": "", "decision": ""}, cfg2)
print("Case 2 (paused):", "__interrupt__" in out2)

out2b = app.invoke(Command(resume=True), cfg2) # human approves
print("Case 2 (resumed):", out2b["decision"])
```

```
Case 1: Auto-approved $120 for 'team lunch'.  
Case 2 (paused): True  
Case 2 (resumed): Approved by human reviewer: $900 for 'conference travel'.
```

Notice this capstone is just Sections 6, 9, 12, and 13 combined -- that's the real skill in LangGraph: each concept is small and composes cleanly with the others.

18. Cheat Sheet / Quick Reference

API	What it does
StateGraph(Schema)	Start building a graph with the given state shape.
graph.add_node(name, fn)	Register a step. fn(state) -> partial update dict.
graph.add_edge(a, b)	Always go from a to b.
graph.add_conditional_edges(a, router, map)	Go from a to whichever node router(state) names.
START, END	Special markers for "begin here" / "run is finished".
graph.compile(checkpointer=...)	Turn the builder into a runnable app.
app.invoke(state, config)	Run the graph once to completion (or to the next interrupt).
app.stream(state, stream_mode=...)	Run the graph, yielding output as each node finishes.
Annotated[list, add_messages]	Reducer: append chat messages instead of overwriting.
Annotated[list, operator.add]	Reducer: append/merge any list-typed field.
ToolNode(tools)	Prebuilt node that executes whatever tools the model just called.
tools_condition	Prebuilt router: "does the last message have tool_calls?"
create_agent(model, tools=...)	Prebuilt full ReAct-style agent (from langchain.agents).
InMemorySaver()	Checkpointers for local dev; use Postgres/SQLite savers in production.
{"configurable": {"thread_id": ...}}	The config that ties a run to a saved conversation.
interrupt(value)	Pause a node and hand value to a human/external system.
Command(resume=...)	Resume a paused graph with the human's answer.
Command(update=..., goto=...)	From inside a node: update state AND choose the next node.
app.get_state(config)	Inspect the current saved state for a thread.

19. Common Pitfalls & Best Practices

- **Forgetting a reducer on a growing field.** If a field should accumulate (chat messages, a running list of results), it needs `add_messages` or `operator.add` -- otherwise every update silently replaces the whole field.
- **Using `InMemorySaver` in production.** It's an in-process dict; a restart loses every conversation. Swap in a persistent checkpointer (Postgres/SQLite) before shipping.
- **Forgetting the checkpointer when using `interrupt()`.** `interrupt()` has nowhere to save the paused state without one, and will error.
- **No exit condition on a cycle.** A conditional edge that never routes to END will loop forever (or until a recursion-limit error). Always make sure the router's condition can eventually become true.
- **Reaching for a hand-built agent loop before checking `create_agent`.** If you just need standard "call tools until done", the prebuilt agent (Section 10) will save you the boilerplate in Section 9 -- hand-roll only when you need custom routing.
- **Import drift.** LangGraph moves fast; `create_react_agent` moving to `langchain.agents.create_agent` in v1.0 is a good example. When something you read online doesn't match what you installed, check `pip show langgraph` and the official migration guide before assuming your code is wrong.

References

- LangGraph documentation (official): <https://langchain-ai.github.io/langgraph/>
- LangGraph v1 migration guide: <https://docs.langchain.com/oss/python/migrate/langgraph-v1>
- langgraph-supervisor package reference: <https://reference.langchain.com/python/langgraph-supervisor>
- LangChain blog -- "Plan-and-Execute Agents" (background on agent architectures): <https://www.langchain.com/blog/planning-agents>

Every code example in this document was executed against langgraph 1.2.11 and langchain 1.6.0 in a clean environment before inclusion here; the printed output blocks are the real output of that run (fake, deterministic chat models stood in for a real API so every example runs with no API key). If your installed version differs, run `pip show langgraph` and check the migration guide above for what changed.